

Demo: Table Design - Compression

Summary

Checked default Redshift compression encoding using `pg_table_def`, analyzed recommended encodings with `ANALYZE COMPRESSION`, and created a custom table using `ZSTD` compression for selected columns to optimize storage and performance.

Check Existing Compression Encoding

Target table: `usersauto`

Query Compression Information

The screenshot shows the AWS Redshift Query Editor v2 interface. The left sidebar contains navigation options: Editor, Queries, Notebooks, Charts, History, and Scheduled queries. The main editor area displays a SQL query:

```
1 -- compression
2 select "column", type, encoding
3 from pg_table_def where tablename = 'usersauto' and schemaname = 'table_design_demo';
4
5 ANALYZE COMPRESSION usersauto;
6
7 create table users_custom_compression(
8   userid integer not null distkey sortkey,
9   username char(8) encode zstd,
10  firstname varchar(30) encode zstd,
11  lastname varchar(30) encode zstd,
12
```

Below the query editor, the 'Result 1 (18)' table is displayed, showing the output of the query. The table has three columns: column, type, and encoding.

column	type	encoding
userid	integer	none
username	character(8)	lzo
firstname	character varying(30)	lzo
lastname	character varying(30)	lzo
city	character varying(30)	lzo
state	character(2)	lzo
email	character varying(100)	lzo
phone	character(14)	lzo
birthdate	timestamp	none

```
SELECT tablename,s
       "column",
       encoding
FROM pg_table_def
WHERE tablename = 'usersauto';
```

Observations

Dist Key / Sort Key `userid` has: `No Encoding`

Reason

Redshift does not compress:

- Distribution keys
- Sort keys

Default Character Encoding

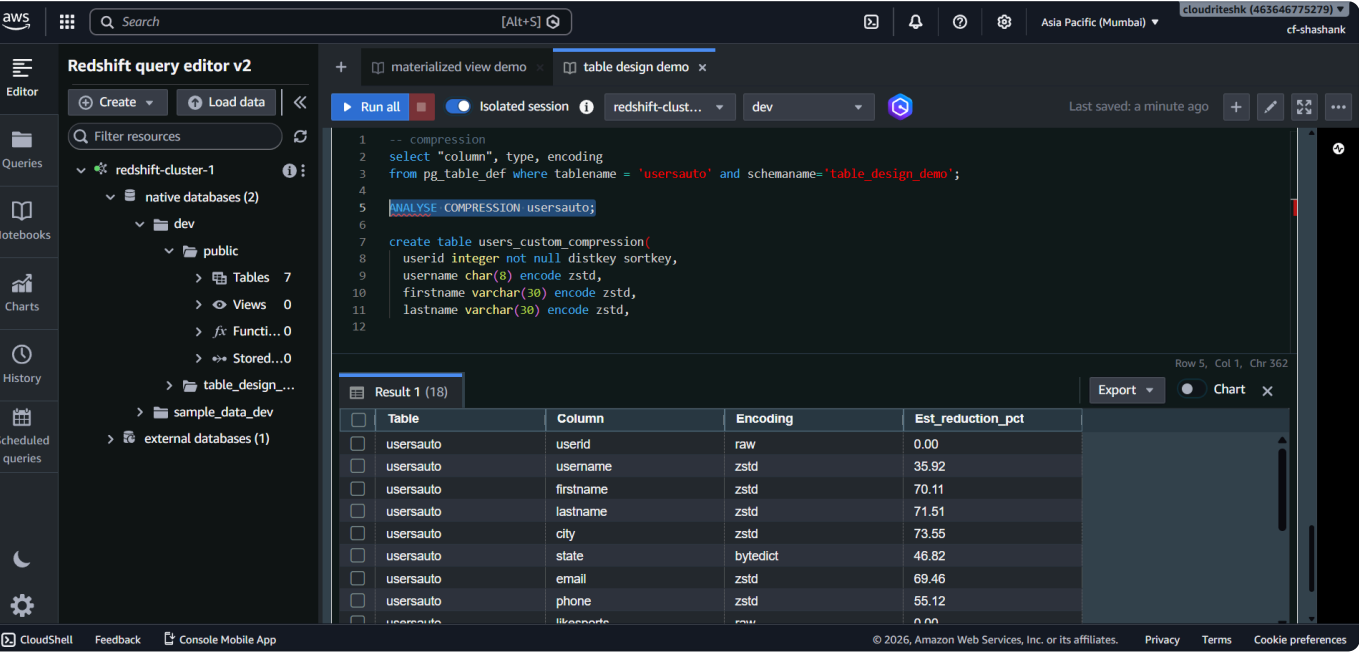
Columns like:

- username
- firstname
- lastname
- phone

used: `LZO` encoding by default.

Analyze Compression Recommendations

ANALYZE COMPRESSION Command



```
ANALYZE COMPRESSION usersauto;
```

Output Includes

Table column, Recommended encoding, Estimated storage reduction

Recommendation Observed

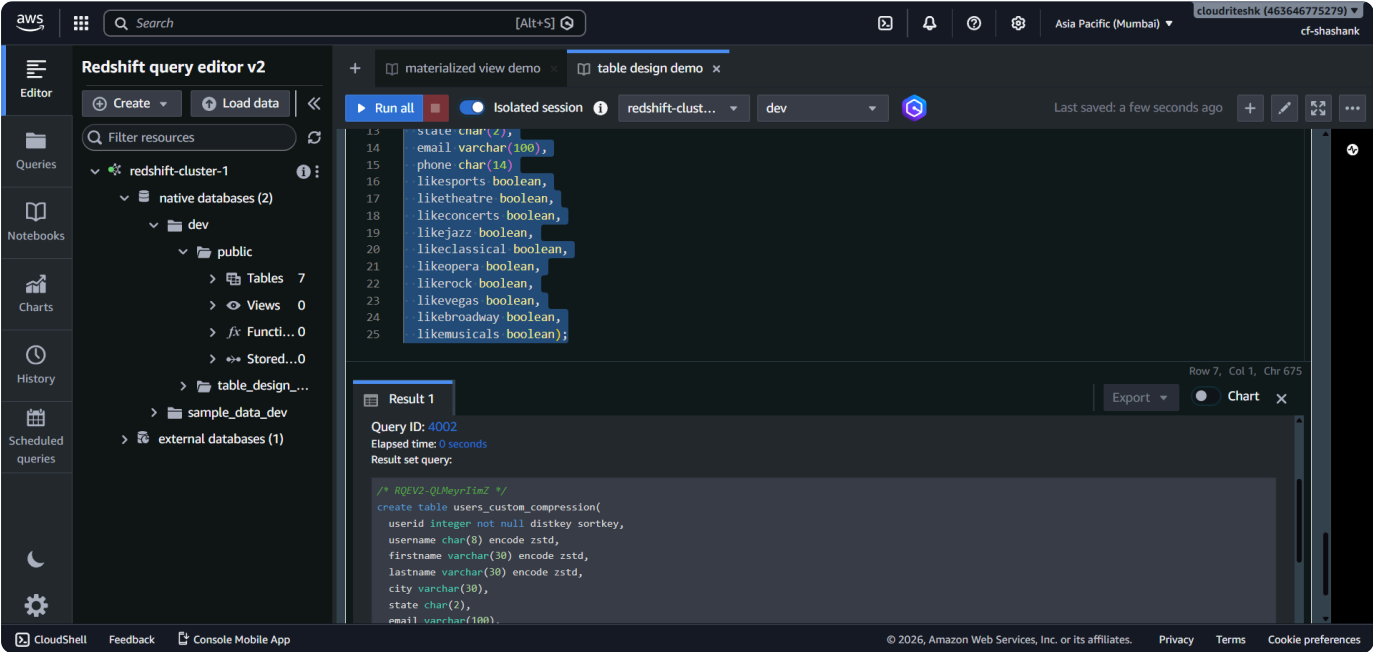
For columns like:

- firstname
- lastname
- username

Redshift recommended: `ZSTD` encoding.

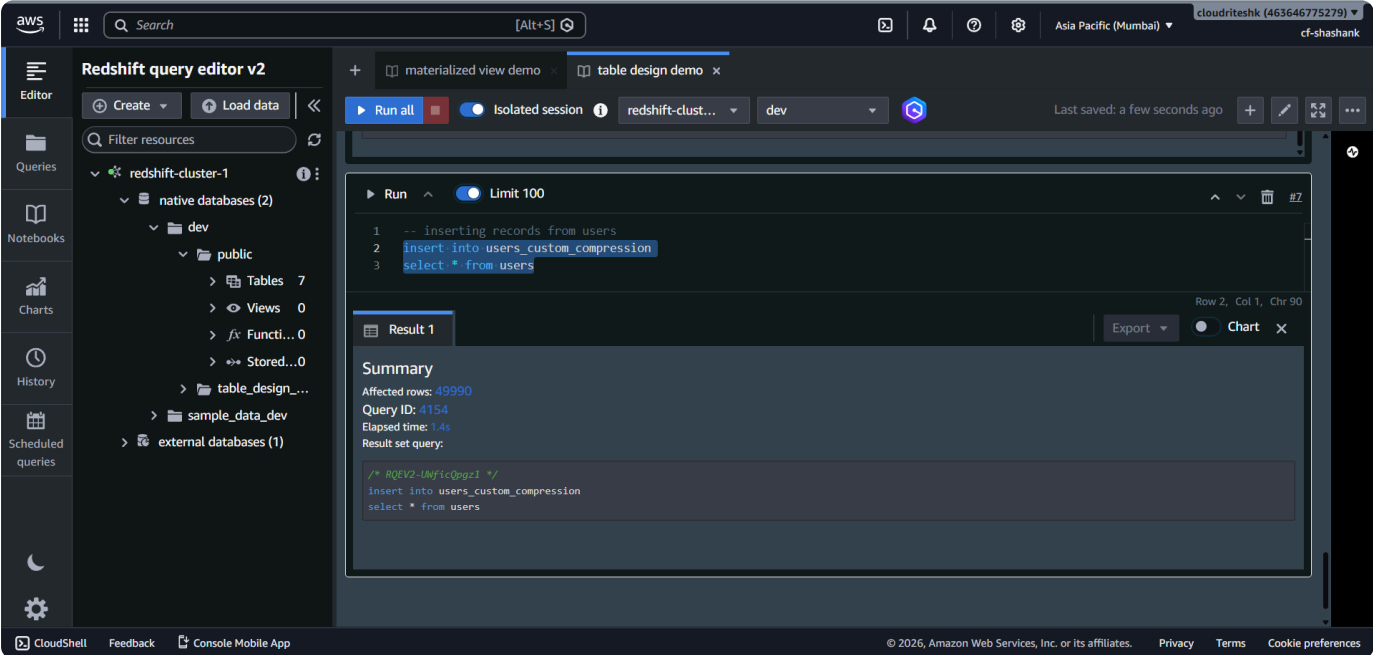
Create Table with Custom Compression

Create Table



```
CREATE TABLE users_custom_encoding (  
  userid INTEGER,s  
  username VARCHAR ENCODE zstd,  
  firstname VARCHAR ENCODE zstd,  
  lastname VARCHAR ENCODE zstd,  
  city VARCHAR  
);
```

Insert Data



```
INSERT INTO users_custom_encoding  
SELECT userid,  
  username,  
  firstname,
```

```
        lastname,  
        city  
FROM public.users;
```

Verify Compression Encoding

The screenshot shows the AWS Redshift Query Editor v2 interface. The left sidebar contains navigation options: Editor, Queries, Notebooks, Charts, History, and Scheduled queries. The main editor area displays a query: `select "column", type, encoding from pg_table_def where tablename = 'users_custom_compression' and schemaname = 'table_design_demo';`. The query has been executed, and the results are shown in a table with 18 rows. The table has three columns: column, type, and encoding. The results show that the 'users' table uses 'zstd' encoding, while other tables use 'lzo' or 'none'.

column	type	encoding
userid	integer	none
username	character(8)	zstd
firstname	character varying(30)	zstd
lastname	character varying(30)	zstd
city	character varying(30)	lzo
state	character(2)	lzo
email	character varying(100)	lzo
phone	character(14)	lzo
likesports	boolean	none

```
SELECT tablename,s  
       "column",  
       encoding  
FROM pg_table_def  
WHERE tablename = 'users_custom_encoding';
```

Observation

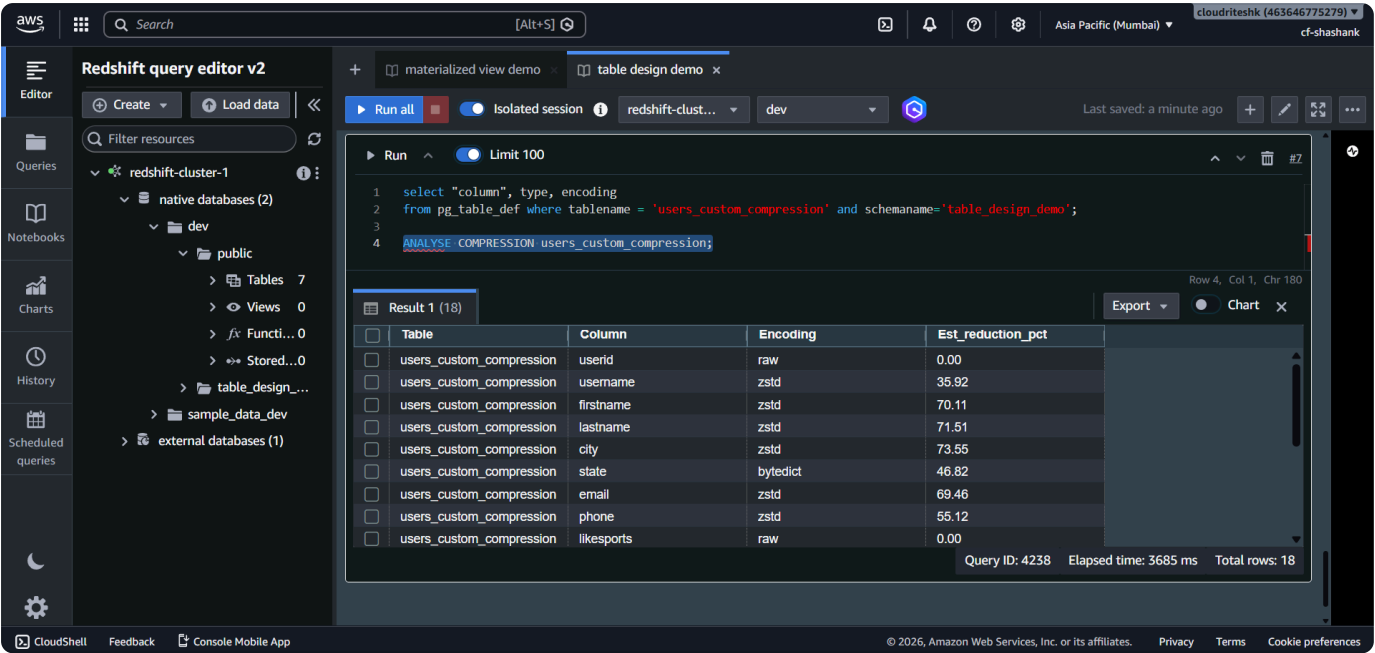
Columns:

- username
- firstname
- lastname

now use: `ZSTD`

Other character columns still use: `LZO` (default Redshift encoding).

Re-Run Compression Analysis



`ANALYZE COMPRESSION users_custom_encoding;`

Observation

No recommendations were given for columns already using: `ZSTD`

Only remaining columns received recommendations.

Key Learning

Compression encoding helps:

- Reduce storage usage
- Improve query performance
- Reduce scan cost

Redshift automatically applies compression, but custom encoding can also be manually defined using:

- `CREATE TABLE`
- `ALTER TABLE`